

Group 11 - Cube^{Cubed}

Team 11: Audrey Garon (asg289), Michelle Mobius (mrm386), Kate Sevigny (kas573)

Robot Design and Strategy Overview

Our robot, “Cubie”, was designed to optimize the physical dimension constraints given by the competition rules. The robot was required to fit within an 8”x8” box prior to the competition and a 12”x12” cylindrical container after it commenced. Our main goal was to collect the most amount of cubes we could within the 12”x12” constraint. To do so, we designed “arms” that were positioned vertically before the competition began and rotated to become horizontal once it began (Appendix C). One arm was positioned at the front of each side of the robot, each connected to a continuous 360° microservo that allowed for rotation downward. This gave our robot leverage to both collect more blocks since the size of the perimeter surrounding the robot increased and to provide a defense mechanism when approaching the other team’s robot. When moving forward, the arms could “hug” the other team’s robot, essentially trapping them and blocking them from collecting anything.

Additionally, we designed a “license plate” positioned on the back of the robot to ensure blocks stayed within the perimeter (Appendix C). This plate prevented blocks from being swept under and out from the robot when it was moving.

We also purchased larger wheels which allowed for more room beneath the robot for blocks to collect, as well (Appendix A). These larger wheels created small gaps between the arms and board beneath the robot, so we also added cardboard flaps to stop blocks from escaping from beneath the front of the robot. Larger wheels also gave our robot a height advantage over the other teams’ robots, because the additional height made it more difficult for them to push us over.

In terms of electronics, we optimized the use of two motors (one for each wheel) and two microservos (one for each arm), wired as seen in Appendix B. During the process of working on all four milestones, we found that the sensors were very finicky and not reliable. Ultimately, we decided to not use them and instead focused on hard-coding the robot so that it could consistently collect the most blocks.

The software design of the robot was hard-coded, allowing for a dependable route for the robot to take each round of the competition. The robot moved at an approximate 45° towards the left side of the board, turned right so it was parallel with the interface between colors, moved forward the length of the board, and then turned and moved towards the edge

we started on. The intention was to retreat in order to maintain the blocks collected and avoid the other robot since we did not utilize code to address running into physical obstacles. The full code is included in Appendix E with a logic flowchart in Appendix D.

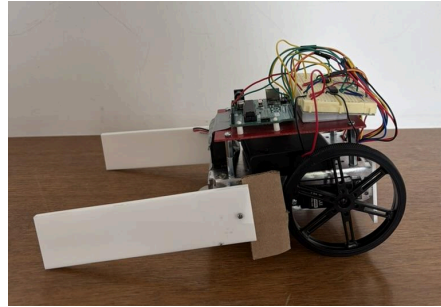


Figure 1: Image of Cubie prior to competition

Design Process Reflection

Our main objection for each milestone and the robot competition was to minimize complexity while maximizing capability. We approached the design process one milestone at a time, focusing specifically on the requirements of each stage before moving forward. While completing each milestone, we thoroughly tested the robot's performance through our code and physical trials to ensure everything functioned properly.

During Milestones One and Two, we quickly assembled Cubie by attaching the wheels, installing standoffs for the circuit board, wiring the wheel motors via H-Bridge, and securing all components to the main chassis. After completing Milestone Two, we observed that the original wheels were too small, so we opted to purchase larger wheels to maximize the available space beneath the robot for cube collection in later milestones. For Milestone Three, we added the color sensor. We were careful to introduce only one new electrical component at a time and thoroughly tested the robot after each addition to ensure functionality.

From a mechanical design perspective, we prioritized modularity by ensuring every major component attached directly to the chassis (our primary structural element). For Milestone Four, we mounted the continuous 360° microsensors directly onto the chassis. We modeled the arm attachment system after the wheel-to-motor interface, designing simple rectangular arms that screwed directly into the servo horns (Appendix C). To ensure proper functionality, we added two nuts between each servo and arm to act as spacers.

We were very iterative throughout the design and testing process, constantly testing through code and making adjustments as needed. We did not face any major roadblocks until Milestone Four, when we realized cubes were slipping through the sides of the arms due to

too much clearance. We fixed this with simple cardboard cutouts secured with duct tape. In Figure 1, all the added components are visible on Cubie before competition.

Competition Analysis

Our robot won four out of seven matches in the round robin portion of the competition, earning third place within our group. We successfully collected blocks each round. In round 3, for instance, we had four blocks over the other team's zero. We rammed straight into them and stopped them from collecting any. The arms of the robot held them in place while ours kept moving, collecting blocks while they had none. Rounds we lost were typically due to the fact that our code was hard-coded and the other teams were more dynamic since theirs relied on sensors. For example, in several rounds, the other teams' robot pushed us intentionally off the board and collected the rest of the blocks. Figure F.3 in Appendix F shows an instance where Cubie was pushed and flipped, resulting in all blocks being lost.

The strength of our robot relied on its mechanical design and the reliability of our code. The combination of the arms with the larger wheels allowed us to maximize the number of blocks we collected and kept. While other teams sometimes would collect many blocks, they would lose them if they turned or moved backwards. Due to the arms and "license plate" on the back, blocks we collected stayed within our perimeter. Because our code was hard-coded, there were no issues with sensors not working and our robot consistently got blocks each round. However, because the code was hard-coded, our robot lacked dynamic awareness of what the other teams' robot did, meaning that if they intercepted our path or our robot, we could not reliably respond well.

Conclusions

If we could do this project over again with the same constraints and budget, we would spend more time working on the code. We are proud of our mechanical design, however because our code could not respond to environmental changes on the board, we did not make it out of the round-robin portion of the tournament. Additionally, we would make the arms of the robot wider, allowing us to collect even more blocks whilst remaining in the geometric constraints. To prevent turning over, we would also add more weight to strengthen the defensive effort.

For students doing the project next year, we would recommend trying to spend more time on the code than the mechanical design. We noticed that robots that progressed further in the competition had a similar mechanical to ours but had better code.

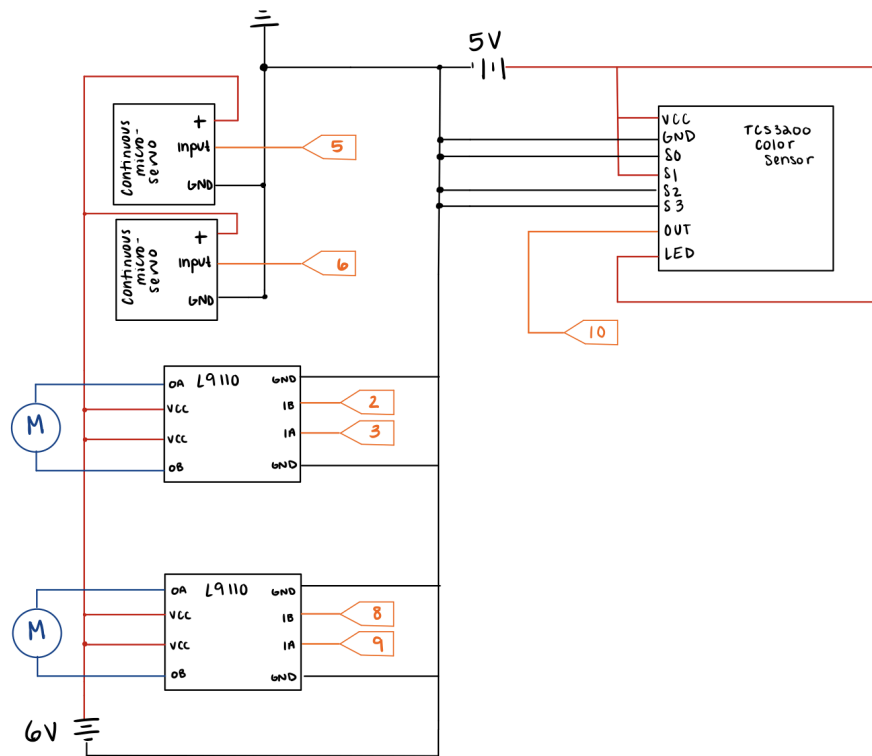
Appendix A - [Bill of Materials]

Part No.	Name	Description	Supplier	Supplier Part No	Quantity	Unit Cost	Total Cost
001	RoboArms	Robot arm (6 in x 1.57 in x 0.25 in) → attaches directly to chassis	RPL	N/A	2	1.26	2.51
002	Acrylic Robo License Plate	Robot back acrylic plate (3.3 in x 2 in x 0.25 in) → attaches directly to chassis	RPL	N/A	1	1.03	1.03
003	90x10 mm diameter wheels	Pololu Wheel for Standard Servo Splines (25T, 5.8mm) - 90×10mm, Black, 2-Pack	Polulu	91864A078	1	9.49	9.49
004	Continuous 360 ° microservo for arms	Continuous 360 ° microservo	MAE 3780 Lab Shop	N/A	2	3	6
005	Acrylic sheet	Acrylic sheet (for license plate and roboarms)	RPL	N/A	1	5	5

Cubie Total Cost: \$ 24.03

Appendix B

Circuit Diagram



Appendix C

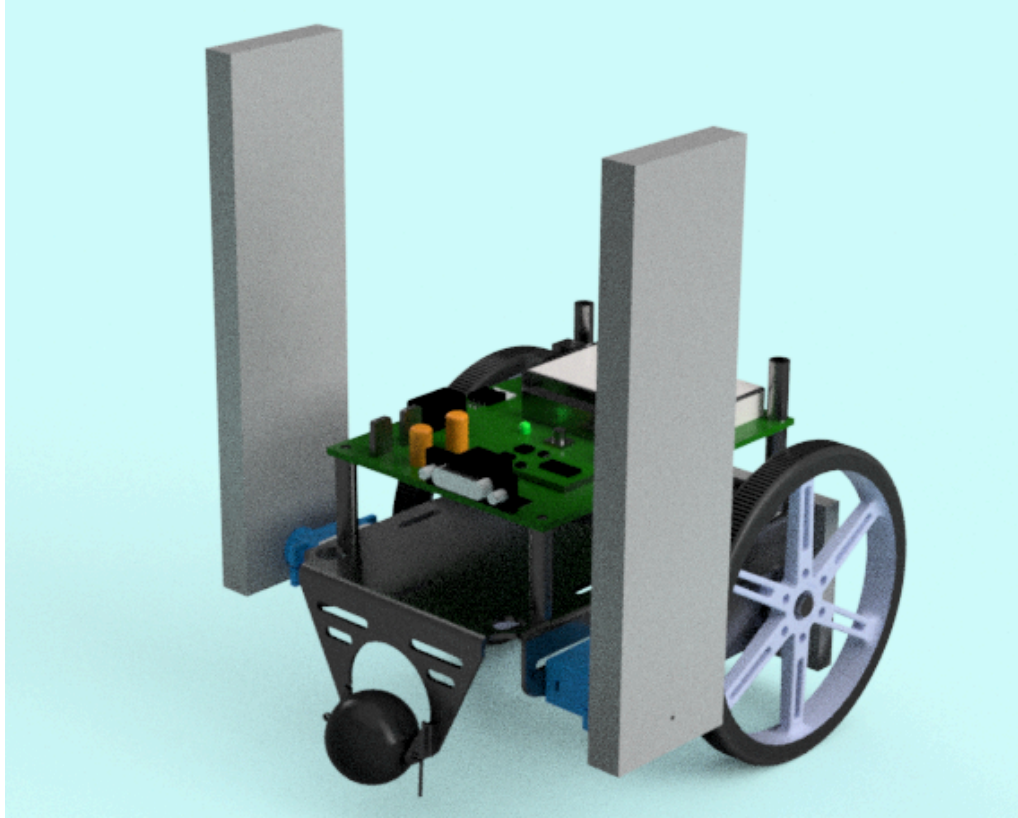


Figure C1: Isotropic view w/ roboarms up

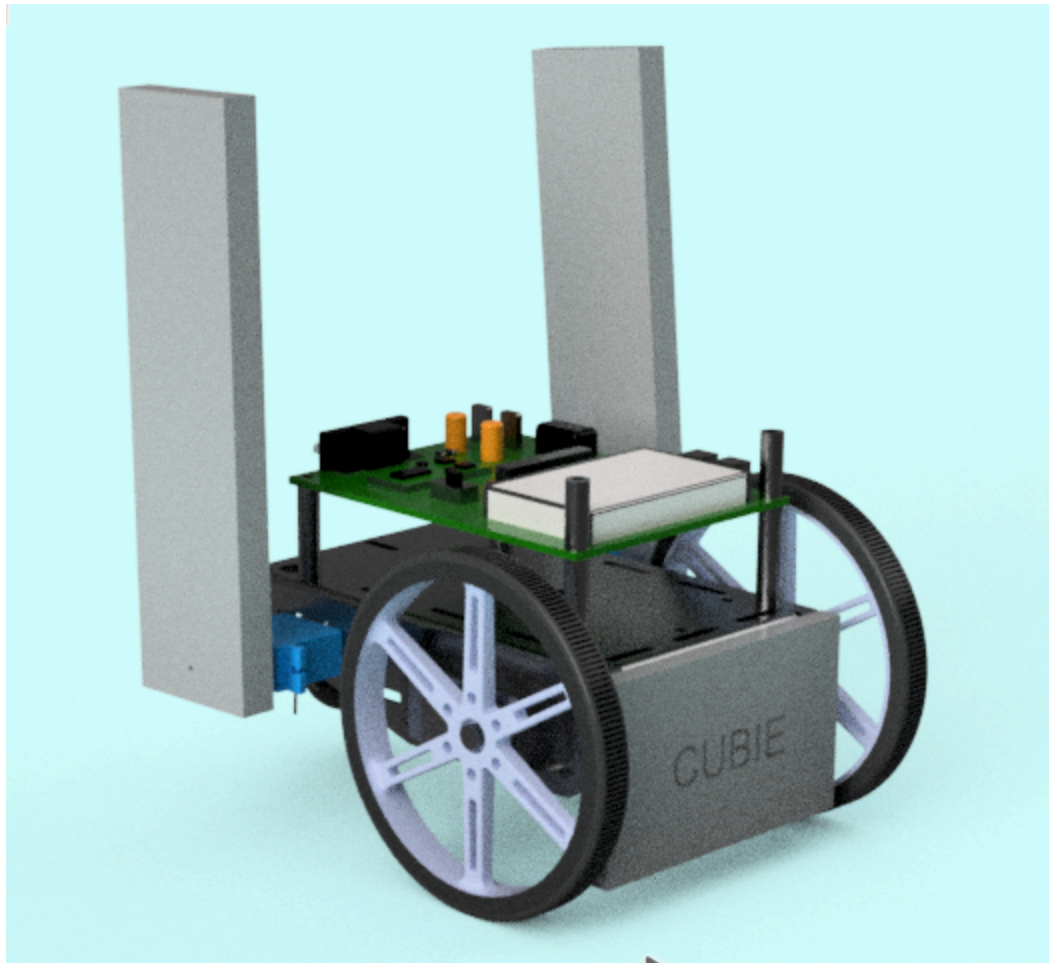


Figure C2: Back view

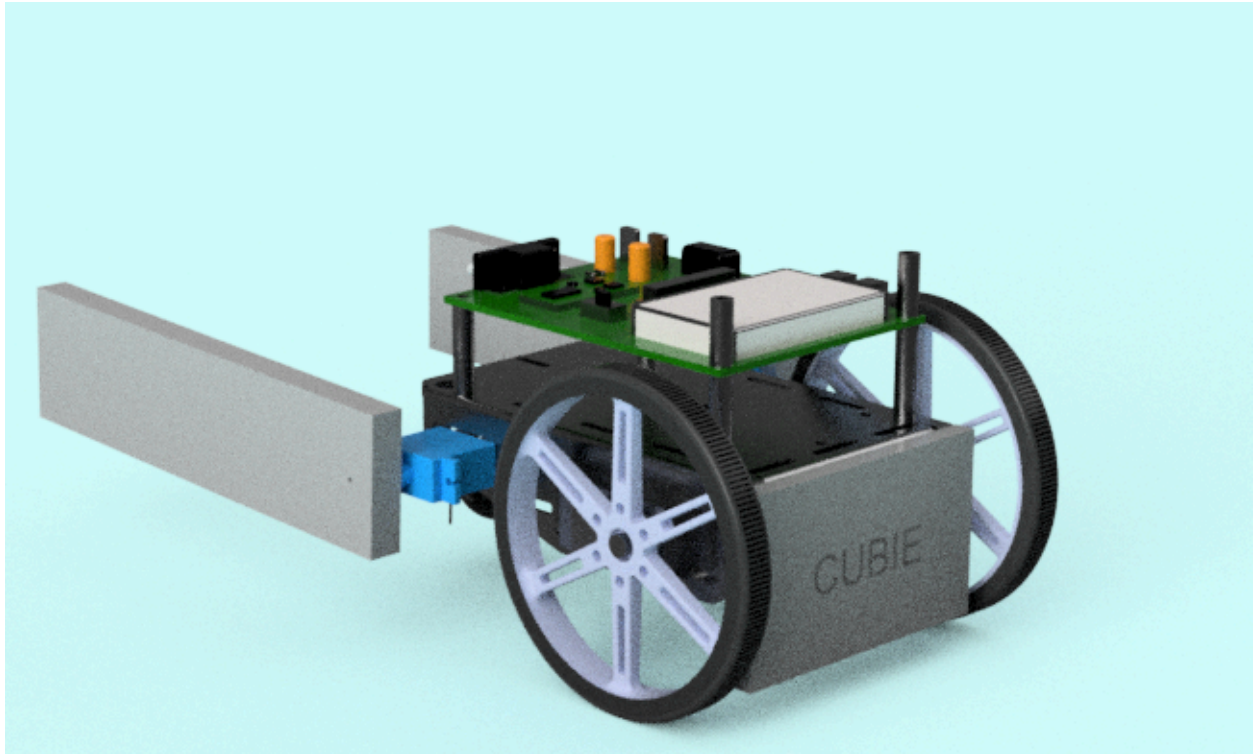
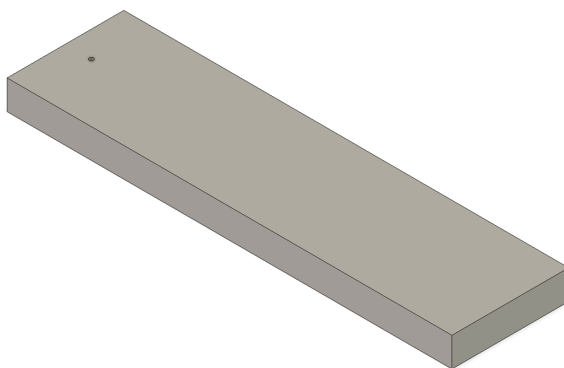


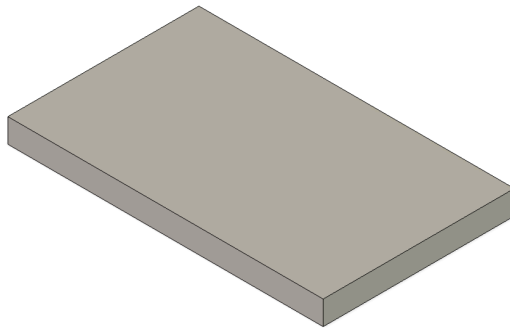
Figure C3: Back view, arms deployed

RoboArms: weight of each arm is $0.71 \text{ N} = 0.16 \text{ lb}$



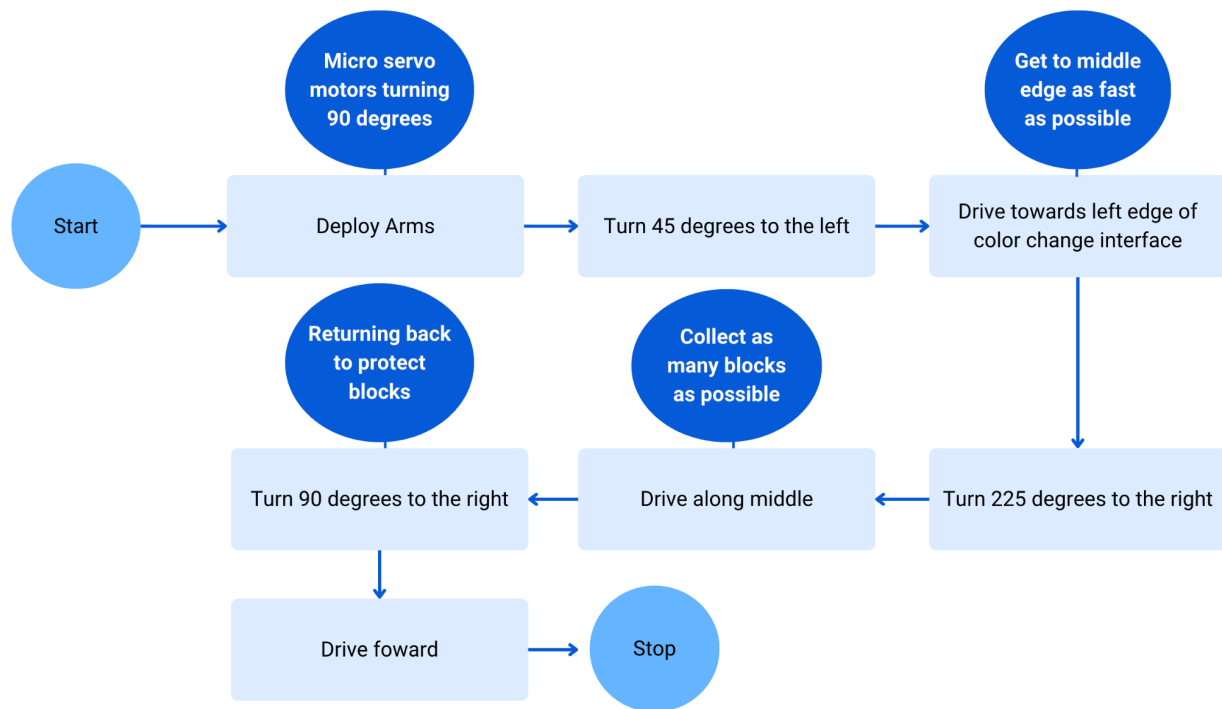
PROPERTIES	
Faces (11)	
Physical Material	Acrylic
Appearance	Plastic - Glossy (Grey)
Bodies (2)	
Area	16254.791 mm ²
Density	0.001 g / mm ³
Mass	72.535 g
Volume	61056.628 mm ³
Physical Material	Acrylic
Appearance	Plastic - Glossy (Grey)

License plate backing: weight of backing is $0.315 \text{ N} = 0.071 \text{ lb}$



PROPERTIES	
Faces (6)	
Physical Material	Acrylic
Appearance	Plastic - Glossy (Grey)
Bodies (1)	
Area	10225.786 mm ²
Density	0.001 g / mm ³
Mass	32.122 g
Volume	27038.656 mm ³
Physical Material	Acrylic
Appearance	Plastic - Glossy (Grey)
Bounding Box	
Center of Mass	41.91 mm, 3.175 mm, ...
Moment of Inertia at Center of Mass (g mm ²)	
Moment of Inertia at Origin (g mm ²)	
Copy To Clipboard	

Appendix D



Appendix E

```
#include <avr/io.h>
#include <util/delay.h>
```

```
// servo pin definitions
```

```

#define SERVO_L_PIN PD5    // left servo connected to PD5
#define SERVO_R_PIN PD6    // right servo connected to PD6
#define SERVO_DDR DDRD    // data direction register for port D
#define SERVO_PORT PORTD   // port D output register

// servo pulse widths in microseconds
#define SERVO_0_DEG 1000   // 0 degrees position
#define SERVO_90_DEG 1500  // 90 degrees position
#define SERVO_180_DEG 2000 // 180 degrees position

// number of 20ms cycles to hold position (about 1 second total)
#define SERVO_HOLD_FRAMES 50

// servo pulse function- sends one 20ms PWM signal to a servo
static void servo_pulse(uint8_t pin, uint16_t pulse_us)
{
    // set pin HIGH for pulse duration
    SERVO_PORT |= (1 << pin);

    uint16_t high_ticks = pulse_us / 10;
    for (uint16_t i = 0; i < high_ticks; i++)
        _delay_us(10);

    // set pin LOW for remainder of 20ms period
    SERVO_PORT &= ~(1 << pin);

    uint16_t low_ticks = (20000 - pulse_us) / 10;
    for (uint16_t i = 0; i < low_ticks; i++)
        _delay_us(10);
}

// set both arms - left arm automatically mirrors right arm
void set_arms(uint16_t right_pulse_us)
{
    // calculate mirrored pulse for left servo
    uint16_t left_pulse_us = (uint16_t)(SERVO_0_DEG + SERVO_180_DEG) - right_pulse_us;

```

```

// repeat pulses to hold position
for (uint8_t i = 0; i < SERVO_HOLD_FRAMES; i++)
{
    servo_pulse(SERVO_R_PIN, right_pulse_us);
    servo_pulse(SERVO_L_PIN, left_pulse_us);
}

// only control right arm
void set_arm_right(uint16_t pulse_us, uint8_t frames)
{
    for (uint8_t i = 0; i < frames; i++)
        servo_pulse(SERVO_R_PIN, pulse_us);
}

// only control left arm
void set_arm_left(uint16_t right_equivalent_pulse_us, uint8_t frames)
{
    // convert right-side angle to left-side pulse
    uint16_t left_pulse_us = (uint16_t)(SERVO_0_DEG + SERVO_180_DEG) -
    right_equivalent_pulse_us;

    for (uint8_t i = 0; i < frames; i++)
        servo_pulse(SERVO_L_PIN, left_pulse_us);
}

// initialize servo pins as outputs
// =====
void servo_init(void)
{
    SERVO_DDR |= (1 << SERVO_L_PIN) | (1 << SERVO_R_PIN); // set pins as output
    SERVO_PORT &= ~(1 << SERVO_L_PIN) | (1 << SERVO_R_PIN); // start LOW
}

// motor control functions
void drive_forward(void)
{

```

```

    PORTB = 0b00000010; // set motor direction
    PORTD = 0b00001000;
}

void drive_backward(void)
{
    PORTB = 0b00000001;
    PORTD = 0b00000100;
}

void drive_right(void)
{
    PORTB = 0b00000001;
    PORTD = 0b00001000;
}

void drive_left(void)
{
    PORTB = 0b00000010;
    PORTD = 0b00000100;
}

void stop(void)
{
    PORTB = 0b00000000; // stop motors
    PORTD = 0b00000000;
}

int main(void)
{
    // set motor pins as output (PB0, PB1, PD2, PD3)
    DDRB = 0b00000011;
    DDRD = 0b01101100; // PD2, PD3, PD5, PD6 as outputs

    servo_init();    // initialize servos

    // deploy arms once at startup

```

```

set_arms(SERVO_0_DEG); // move to known starting position
_delay_ms(10);

set_arms(SERVO_90_DEG); // move arms outward
_delay_ms(10);

// main driving loop
while (1)
{
    drive_left();
    _delay_ms(250);

    drive_forward();
    _delay_ms(2500); //move forward

    drive_right();
    _delay_ms(700); //turn right

    drive_forward();
    _delay_ms(2300); //move forward

    drive_right();
    _delay_ms(500);

    drive_forward();
    _delay_ms(2000);

    stop();      // stop at end
    _delay_ms(1000000000); // long delay (basically stop forever)
}
}

```

Appendix F

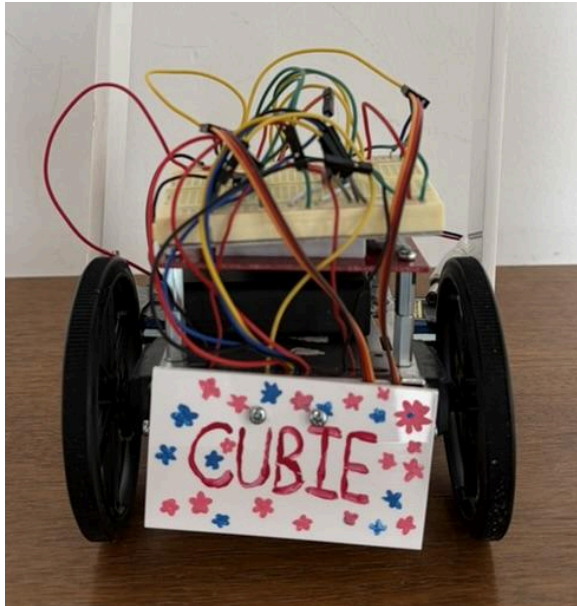


Figure F.1: Back view

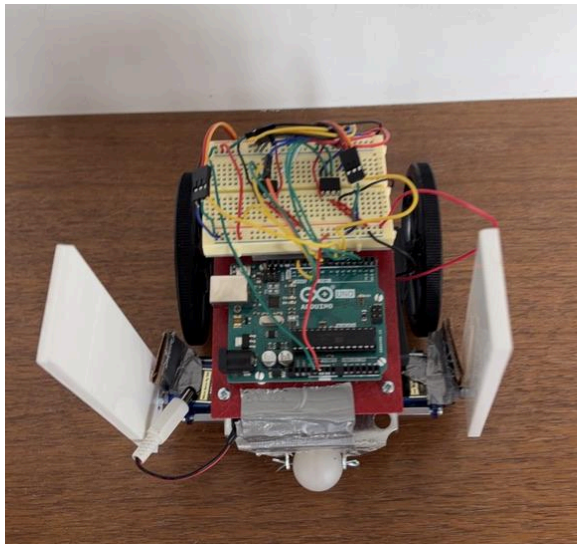


Figure F.2: Top View

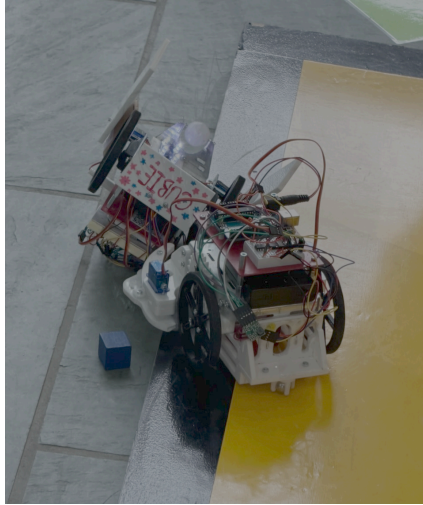


Figure F.3: Competition